# Pruebas automatizadas de SnackUp

## Informe de resultados y evidencias

20 de octubre de 2026

Se ejecutaron cinco pruebas automatizadas sobre las pantallas de detalle de producto y carrito de SnackUp. **Las cinco aprobaron**: selección de cantidades, límite mínimo, cálculo decimal, rechazo de pedidos de distintas tiendas y creación de un pedido válido.

### Proyecto y entorno

Framework Flutter 3.32.4 · Dart 3.8.1 · Windows
Herramienta flutter_test · Firebase Auth y Firestore simulados
Rama codex/pruebas-snackup-informe · Commit del código 46692b0
Archivo test/snackup_test.dart

Repositorio: github.com/Gabino-RG/SnackUP-core

### Resumen de resultados

| Caso | Comprobación | Resultado |
|---|---|---|
| CP-01 | Aumentar la cantidad | **Aprobado** |
| CP-02 | Respetar la cantidad mínima | **Aprobado** |
| CP-03 | Calcular un total con decimales | **Aprobado** |
| CP-04 | Rechazar productos de varias tiendas | **Aprobado** |
| CP-05 | Crear un pedido y vaciar el carrito | **Aprobado** |

### Alcance de la evidencia

Cada caso incluye los pasos AAA y el evento individual testDone emitido por Flutter. El valor result success confirma que sus comprobaciones terminaron sin fallos. Son pruebas de widgets con servicios simulados; no comprueban las reglas de seguridad ni la conexión de Firebase real.

# CP-01 Aumentar la cantidad

**RESULTADO APROBADO**

**Código probado**
ProductDetailScreen y método _increment

## Datos de entrada

Precio $25.00, stock 10 y cantidad inicial 1.

## Procedimiento AAA

**ARRANGE Preparar**
Abrir la pantalla de detalle con un producto disponible y un usuario simulado autenticado.

**ACT Actuar**
Pulsar una vez el botón con el icono add_rounded.

**ASSERT Comprobar**
Comprobar que la cantidad visible es 2 y que el botón muestra Añadir 2 al Carrito - $50.00.

## Resultado obtenido

La cantidad aumentó a 2 y el total visible fue $50.00.

## Evidencia individual de ejecución

```
Prueba: CP-01: aumentar cantidad de 1 a 2
testID: 3
result: success
skipped: false
type: testDone
time: 11569 ms
```

Extracto de evidencia_pruebas.jsonl. El identificador enlaza el inicio de esta prueba con su resultado; skipped false confirma que se ejecutó. El campo time indica milisegundos desde el inicio de la ejecución.

## Comprobaciones del archivo de pruebas

Las verificaciones se ejecutan mediante expect dentro del archivo test/snackup_test.dart. La aprobación corresponde al conjunto de comprobaciones descrito en ASSERT, incluida la consulta de datos simulados en los casos CP-04 y CP-05.

Ubicación del caso: línea 60 del archivo de pruebas.

# CP-02 Respetar la cantidad mínima

**RESULTADO APROBADO**

**Código probado**
ProductDetailScreen y método _decrement

## Datos de entrada

Precio $25.00, stock 10 y cantidad inicial 1.

## Procedimiento AAA

**ARRANGE Preparar**
Abrir el detalle con la cantidad predeterminada de una unidad.

**ACT Actuar**
Pulsar dos veces el botón con el icono remove_rounded.

**ASSERT Comprobar**
Comprobar que la cantidad permanece en 1 y que el botón muestra un total de $25.00.

## Resultado obtenido

La cantidad permaneció en 1 después de ambos intentos; el total siguió en $25.00.

## Evidencia individual de ejecución

```
Prueba: CP-02: la cantidad no baja de uno
testID: 4
result: success
skipped: false
type: testDone
time: 11772 ms
```

Extracto de evidencia_pruebas.jsonl. El identificador enlaza el inicio de esta prueba con su resultado; skipped false confirma que se ejecutó. El campo time indica milisegundos desde el inicio de la ejecución.

## Comprobaciones del archivo de pruebas

Las verificaciones se ejecutan mediante expect dentro del archivo test/snackup_test.dart. La aprobación corresponde al conjunto de comprobaciones descrito en ASSERT, incluida la consulta de datos simulados en los casos CP-04 y CP-05.

Ubicación del caso: línea 71 del archivo de pruebas.

# CP-03 Calcular un total con decimales

**RESULTADO APROBADO**

**Código probado**
ProductDetailScreen y cálculo price * _quantity

## Datos de entrada

Precio unitario $18.50, stock 10 y cantidad inicial 1.

## Procedimiento AAA

**ARRANGE Preparar**
Abrir el detalle del producto con precio decimal.

**ACT Actuar**
Pulsar dos veces el botón de aumentar para seleccionar tres unidades.

**ASSERT Comprobar**
Comprobar que el botón muestra Añadir 3 al Carrito - $55.50.

## Resultado obtenido

El total visible fue $55.50 para tres unidades de $18.50.

## Evidencia individual de ejecución

```
Prueba: CP-03: total con precio decimal
testID: 5
result: success
skipped: false
type: testDone
time: 11964 ms
```

Extracto de evidencia_pruebas.jsonl. El identificador enlaza el inicio de esta prueba con su resultado; skipped false confirma que se ejecutó. El campo time indica milisegundos desde el inicio de la ejecución.

## Comprobaciones del archivo de pruebas

Las verificaciones se ejecutan mediante expect dentro del archivo test/snackup_test.dart. La aprobación corresponde al conjunto de comprobaciones descrito en ASSERT, incluida la consulta de datos simulados en los casos CP-04 y CP-05.

Ubicación del caso: línea 83 del archivo de pruebas.

# CP-04 Rechazar productos de varias tiendas

**RESULTADO APROBADO**

**Código probado**
CartScreen y método _placeOrder

## Datos de entrada

Usuario alumno_prueba; sándwich de tienda_A a $25.00 por 2 unidades y agua de tienda_B a $15.00 por 1 unidad. Total $65.00.

## Procedimiento AAA

**ARRANGE Preparar**
Crear el perfil y los dos productos en Firestore simulado. Abrir el carrito y esperar su carga.

**ACT Actuar**
Pulsar Confirmar Pedido.

**ASSERT Comprobar**
Verificar el mensaje de varias tiendas, que orders esté vacío y que los dos productos sigan en el carrito.

## Resultado obtenido

Apareció el mensaje de rechazo. No se creó ningún pedido y se conservaron ambos productos.

## Evidencia individual de ejecución

```
Prueba: CP-04: rechazar productos de varias tiendas
testID: 6
result: success
skipped: false
type: testDone
time: 12521 ms
```

Extracto de evidencia_pruebas.jsonl. El identificador enlaza el inicio de esta prueba con su resultado; skipped false confirma que se ejecutó. El campo time indica milisegundos desde el inicio de la ejecución.

## Comprobaciones del archivo de pruebas

Las verificaciones se ejecutan mediante expect dentro del archivo test/snackup_test.dart. La aprobación corresponde al conjunto de comprobaciones descrito en ASSERT, incluida la consulta de datos simulados en los casos CP-04 y CP-05.

Ubicación del caso: línea 94 del archivo de pruebas.

# CP-05 Crear un pedido y vaciar el carrito

**RESULTADO APROBADO**

**Código probado**
CartScreen y método _placeOrder

## Datos de entrada

Usuario alumno_prueba; dos sándwiches de $25.00 y un agua de $15.00, todos de tienda_A. Pago Efectivo y sin horario programado.

## Procedimiento AAA

**ARRANGE Preparar**
Crear el perfil con número de control 1234 y los dos documentos del carrito en Firestore simulado.

**ACT Actuar**
Pulsar Confirmar Pedido y esperar las actualizaciones.

**ASSERT Comprobar**
Verificar un único pedido con tienda_A, usuario alumno_prueba, estado pending, total $65.00, pago Efectivo, horario nulo y dos elementos con cantidades 2 y 1. Verificar carrito vacío y mensaje de éxito.

## Resultado obtenido

Se creó un pedido con los datos esperados. El carrito quedó vacío y apareció ¡Pedido realizado con éxito!.

## Evidencia individual de ejecución

```
Prueba: CP-05: crear pedido y vaciar carrito
testID: 7
result: success
skipped: false
type: testDone
time: 12784 ms
```

Extracto de evidencia_pruebas.jsonl. El identificador enlaza el inicio de esta prueba con su resultado; skipped false confirma que se ejecutó. El campo time indica milisegundos desde el inicio de la ejecución.

## Comprobaciones del archivo de pruebas

Las verificaciones se ejecutan mediante expect dentro del archivo test/snackup_test.dart. La aprobación corresponde al conjunto de comprobaciones descrito en ASSERT, incluida la consulta de datos simulados en los casos CP-04 y CP-05.

Ubicación del caso: línea 108 del archivo de pruebas.

# Reproducción y evidencia completa

## Cómo repetir las cinco pruebas

Abrir una terminal en la carpeta SnackUP-core y ejecutar:

```
git switch codex/pruebas-snackup-informe
flutter pub get
flutter test test/snackup_test.dart --reporter expanded
```

Para registrar evidencia individual en formato estructurado:

```
flutter test test/snackup_test.dart --machine > evidencia_pruebas.jsonl
```

El archivo de pruebas usa fake_cloud_firestore 2.5.2 y firebase_auth_mocks 0.13.0.

## Salida de la ejecución inicial

```
00:00 +0: CP-01: aumentar cantidad de 1 a 2
00:03 +1: CP-02: la cantidad no baja de uno
00:03 +2: CP-03: total con precio decimal
00:04 +3: CP-04: rechazar productos de varias tiendas
00:04 +4: CP-05: crear pedido y vaciar carrito
00:05 +5: All tests passed!
```

## Comprobación final del registro estructurado

```
{"success": true, "type": "done", "time": 12939}
```

Los cinco eventos individuales reportan success y el cierre global también reporta success true. Resultado global: 5 aprobadas, 0 fallidas y 0 omitidas.

## Archivos de evidencia

test/snackup_test.dart contiene el código AAA.
resultados_pruebas.txt contiene la salida legible de la primera ejecución.
evidencia_pruebas.jsonl contiene los eventos de la ejecución de verificación.

Huella SHA256 del registro estructurado:
77c4310577c4ce1414caa8fca27fb62fce0e462d25e2fbfc75932a9ce04bdace

## Conclusión

Los cinco comportamientos evaluados cumplieron los resultados esperados en el entorno simulado. La evidencia permite identificar y repetir cada prueba. Esta aprobación se limita a los casos y al código revisado; no representa una certificación de toda la aplicación.